

---

# RailKeeper2 Projektstatus

Stand: 10. Mai 2026

Repository: <https://github.com/ichwars/RailKeeper2>

Aktueller Commit: 4b32456 Add actionable overview insights

## Kurzfazit

RailKeeper2 ist aktuell ein lokaler, produktionsnaher Bestandshalter für Modellbahn-Fahrzeuge. Der Schwerpunkt liegt bewusst auf Fahrzeugen, Stammdaten, Import/Export, Artikeldaten-Websuche, Bildern, Dokumenten, Wartung, Digitalfunktionen und CV-Verwaltung. Zubehör ist aus dem aktuellen Funktionsumfang entfernt und bleibt vorerst außerhalb des Scopes.

## Implementiert und nutzbar

### Grundlage und Sicherheit

- Go-Backend mit SQLite und React-Frontend als eine lokal auslieferbare Anwendung.
- Erstinstallation ohne Standardzugang.
- Login, Logout, serverseitige Sessions, Rollenbasis und CSRF-Schutz.
- Argon2id-Passwort-Hashing.
- Sicherheitsheader, SameSite-Cookies und optional sichere Cookies für HTTPS.
- Basis-Rate-Limiting für Login und Setup.
- Audit-Log für Setup, Login/Logout und Fahrzeugereignisse.
- OpenAPI-Vertrag in `openapi/railkeeper.yaml`.
- Docker-Compose-Betrieb mit persistentem Datenverzeichnis.

### Navigation, Design und Bedienung

- Moderne helle und dunkle Darstellung.
- Responsive Layouts für Desktop und Mobile.
- Mobile Navigation mit Hamburger-Menü.
- Menüpunkte: Übersicht, Bestand, Import/Export, Einstellungen.
- Icons statt textlastiger Aktionen an vielen Stellen.
- Karten- und Tabellenansicht im Bestand.
- Sortierbare Bestandstabelle.

### Übersicht

- Cockpit mit Gesamtbestand, Digitalisierungsgrad, Listenwert und Wartungsstatus.

- Bestandsmix nach Kategorien.
- Datenqualität mit Bilder-, Decoder-, Artikelnummer-, EAN- und Dokumentationsquote.
- Hersteller-Ranking.
- Wartungsradar für fällige und kommende Arbeiten.
- Automatisch abgeleitete Handlungsbedarfe.
- Schnellaktionen zu Bestand, Import/Export und Stammdaten.

## Fahrzeuge

- Fahrzeuge anlegen, anzeigen, bearbeiten und löschen.
- Automatische Inventarnummern mit konfigurierbarem Schema.
- Eindeutigkeitsprüfung und Änderungshistorie für Inventarnummern.
- Modellfelder wie Hersteller, Artikelnummer, Bezeichnung, Spurweite, Epoche, Bahngesellschaft, Kategorie, Gattung, Beschreibung, Baureihe und Fahrzeugnummer.
- Technische Detailfelder wie Länge, Gewicht, Farbe, Beschriftung, Beladung, Inneneinrichtung, Achsen, Haftreifen, Radsatz, Stromaufnahme, Adapter, Kupplung, Antrieb, Fahrlicht, Beleuchtung, Soundgenerator, Rauchgenerator und Zusatzinformationen.
- Schiebeschalter steuern, ob zugehörige Eingabefelder aktiv sind.
- Kupplung vorne gleich hinten wird berücksichtigt.

## Stammdaten

- Bearbeitbare Stammdaten für Hersteller, Kategorien, Gattungen, Epochen, Spurweiten, Bahngesellschaften und Symbole.
- Kategorie/Gattung-Abhängigkeiten für die Fahrzeugfassung.
- Importierte Grunddaten aus den bereitgestellten Tabellen und Modellbau-Wiki-Herstellerdaten.
- Hersteller können Webseite und Nenngroesse/Spurweite enthalten.
- Separater Stammdaten-Import/-Export als JSON.

## Artikeldaten-Websuche

- Artikeldaten-Websuche als Kernfeature.
- Suchmuster für Hersteller, Artikelnummer, Bezeichnung, Spurweite und EAN/Barcode.
- DuckDuckGo-basierte Suche ohne Google-Abhängigkeit.
- Herstellerseiten werden bevorzugt bewertet.
- Barcode-Suche nutzt aktuell gezielt die EAN, weil das in Tests bessere Treffer lieferte.
- Ergebnisse werden feldweise als Vorschlag angezeigt.
- Nutzer entscheidet explizit, welche Felder übernommen werden.
- Bestehende Daten werden nicht ungefragt überschrieben.
- Konflikte werden hervorgehoben.

- 
- Quelle/URL wird gespeichert.
  - Quellen werden lesbarer als Seiten-/Shopname angezeigt.
  - Gefundene Bilder können angezeigt, zwischengespeichert und dem Fahrzeug hinzugefügt werden.
  - Vorbereitete Adapter-Struktur für spätere Quellen.
  - Timeout- und Fehlerhandling vorhanden.
  - Artikeldaten-Websuche ist in den Einstellungen abschaltbar.

## **Bilder, QR-Code und Beilagen**

- Mehrere Bilder pro Fahrzeug.
- Hauptbild-Auswahl.
- Sortierung, Vorschau und Originalgrößen-Anzeige.
- Bildquelle öffnen.
- Upload lokaler Bilder.
- Automatische Thumbnails für JPG, JPEG, PNG und WebP.
- QR-Code pro Fahrzeug mit PNG/SVG-Download und Druckansicht.
- Fahrzeugbeilagen mit Kategorie, Bemerkung, Dateigröße, MIME-Typ, Download und PDF-Ansicht im Browser.
- Ausführbare Dateien werden standardmäßig blockiert.
- Uploads sind in Backups eingeschlossen.

## **Wartung und Zustand**

- Wartungshistorie pro Fahrzeug.
- Geplante und durchgeführte Wartungen.
- Reparaturen, Umbauten, Superungen, Reinigung, Schmierung, Decoder-Einbau und Ersatzteilausch.
- Zustand, Fälligkeitsdatum, Kosten und Notizen.
- Wartungsdaten erscheinen im Dashboard.
- Beilagen/Bilder können fachlich an Wartungsvorgänge anschließen.

## **Digitalfunktionen, CVs und Decoderdateien**

- Funktionstasten F0 bis F31.
- Funktionsname, Symbol, Funktionstyp, Moment-/Dauerfunktion und Richtungsabhängigkeit.
- Symbolstammdaten sind bearbeitbar.
- Icon-basierte Darstellung.
- CV-Werte mit CV-Nummer, Wert, Beschreibung, Kategorie und Decoderprofil.
- Import-/Export-Preview für CV-Daten.
- CV-Änderungshistorie.

- 
- CV-Dateien können gespeichert werden.
  - ESU/LokProgrammer-Projektdateien können als Decoderdateien abgelegt werden.
  - ESU-Metadaten werden soweit möglich angezeigt.

## Import, Export, Backup und Druck

- Fahrzeuglisten-Import für CSV, TSV, XLSX, XLS, ODS und JSON.
- Datei wird ausgelesen, bewertet und vor dem Speichern zeilenweise geprüft.
- Manuelles Mapping für bekannte und unbekannte Tabellenköpfe.
- Update-Vorschau mit Feldvergleich.
- Sicherer Duplikat-/Update-Modus.
- Fahrzeugexport.
- Bestands-PDF mit Zusammenfassung und Fahrzeugkarten.
- Getrennte PDF-Aktionen für kompakte und detailreichere Ausgabe.
- Backup/Restore für lokale Daten und Uploads mit Kompatibilitätsprüfung.

## Bereinigt

- Zubehör wurde aus dem aktuellen Scope entfernt.
- Alte oder begonnene Zubehör-Nummernschemata wurden bereinigt.
- Umlaute und sichtbare ASCII-Ersatztexte wurden in den relevanten UI-Bereichen korrigiert.
- Metadaten- und Quellenhinweise in den Stammdaten wurden nutzerfreundlicher reduziert.
- README und Projektdokumentation wurden auf den aktuellen Funktionsumfang gebracht.

## Noch offen

### Nächste sinnvolle Arbeitspunkte

- Detailumfang der Bestands-PDFs gemeinsam festlegen, z. B. Kurzliste, Versicherungs-/Wertliste, Detailblatt je Fahrzeug.
- Übersicht weiter ausbauen, z. B. Wertentwicklung, Datenluecken nach Kategorie, Wartungstrends und Import-Qualität.
- Artikeldaten-Websuche weiter feinjustieren, besonders bei mehrdeutigen Shoptexten, Licht-/Soundbeschreibung und Quellengewichtung.
- Weitere Quellenadapter für Artikeldaten vorbereiten, z. B. Hersteller-spezifische Parser.
- Mehr End-to-End-Tests für Import, Websuche, Uploads, QR-Code und PDF-Ausgabe.
- UI-Feinschliff für sehr kleine Bild-/Suchergebnis-Kacheln und lange Titel.

## Future Updates

- 
- Ersatzteile-Reiter im Fahrzeugdialog.
  - Websuche nach passenden Ersatzteilen mit Bild, Quelle, Artikelnummer, Preis und Update-Funktion.
  - ESU/LokProgrammer-Dateien tiefer auslesen, falls Spezifikation oder geeigneter Export vorliegt.
  - Optionale öffentliche Nur-Lese-Ansicht für QR-Codes.
  - Zubehör erst später und nur nach erneuter fachlicher Entscheidung.
  - Weitere Exportformate, falls benötigt.
  - Optional weitere Suchanbieter. Google wird aktuell bewusst nicht verwendet.

## Aktueller Stand

Der Stand ist lokal gebaut und auf GitHub gesichert. Letzter bekannter sauberer Build:

```
npm.cmd run build
```

Letzter Commit:

```
4b32456 Add actionable overview insights
```